МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Факультет информатики и вычислительной техники
Кафедра вычислительной техники

Лабораторная работа

«Документирование веб-API»

по дисциплине «Технология разработки программного обеспечения»

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ржавин В. В.

Чебоксары, 2026

1. Описание предметной области и документируемой конечной точки

Информационная система «Экскурсионное бюро» автоматизирует продажу и проведение экскурсий: ведение каталога маршрутов, оформление и оплату договоров, выдачу электронных билетов, проведение экскурсий, отчётность гидов и аналитику. Клиентские приложения взаимодействуют с серверной частью через REST API, спроектированный в лабораторной работе 7.

В настоящей работе документируется одна ключевая конечная точка — **POST /contracts** (оформление договора на экскурсию, UC-04). Она выбрана потому, что объединяет данные нескольких сущностей (экскурсия, участники, договор, электронные билеты), содержит обязательные и необязательные поля, возвращает несколько ресурсов и поддерживает идемпотентность и обработку ошибок. Это позволяет показать все основные элементы OpenAPI: параметры, тело запроса, ответы, схемы и примеры.

Конечная точка публичная: договор оформляет клиент, а в ответе сервер выдаёт клиентский токен для последующего доступа к договору и документам. Базовый URL — https://api.excursion-bureau.ru/api/v1; формат данных — JSON (UTF-8), ошибки — Problem Details (RFC 7807).

Входные данные: идентификатор экскурсии, от 1 до 20 участников (ФИО и паспортные данные), контактный email и телефон, согласие на обработку персональных данных. Результат: договор с номером, стоимостью, статусом, сроком оплаты, списком участников и электронными билетами, а также клиентский токен.

2. JSON-структуры данных

В таблице 1 приведены основные конечные точки API, их параметры и ответы. Документируемая конечная точка POST /contracts выделена полужирным.

Метод	Конечная точка	Параметры	Ответы
GET	/routes	search, active, movementType, sort, page, limit	200 Route[]; 400, 422 Problem
GET	/routes/{routeId}	routeId (path)	200 Route; 404 Problem
GET	/excursions	routeId, dateFrom, dateTo, status, sort, page, limit	200 Excursion[]; 422 Problem
GET	/excursions/{excursionId}	excursionId (path)	200 Excursion; 404 Problem
POST	**/contracts**	**Idempotency-Key (header); ContractInput (body)**	**201 ContractResponse; 400, 409, 422 Problem**
GET	/contracts/{contractId}	contractId (path); X-Client-Token / Bearer JWT	200 Contract; 401, 404 Problem
POST	/contracts/{contractId}/payments	contractId (path); PaymentInput (body)	201 Payment; 409, 422 Problem
GET	/contracts/{contractId}/documents	contractId (path); X-Client-Token / Bearer JWT	200 Document[]; 404 Problem
PATCH	/tickets/{ticketId}	ticketId (path); attended (body)	200 Ticket; 403, 404 Problem
POST	/excursions/{excursionId}/reports	excursionId (path); ReportInput (body)	201 Report; 422 Problem

Таблица 1 – Методы, конечные точки, параметры и ответы API

Структура запроса ContractInput приведена в таблице 2.

Поле	Тип данных	Обязательно	Описание	Ограничения
excursionId	string (uuid)	да	Идентификатор экскурсии	UUID из каталога
participants	array<Participant>	да	Участники экскурсии (таблица 3)	от 1 до 20
contactEmail	string (email)	да	E-mail для копии документов	формат email
contactPhone	string	нет	Контактный телефон	10–15 цифр
consentPersonalData	boolean	да	Согласие на обработку ПД	true

Таблица 2 – Структура запроса ContractInput

Вложенный объект Participant (элемент массива participants) приведён в таблице 3.

Поле	Тип данных	Обязательно	Описание	Ограничения
lastName	string	да	Фамилия участника	1–50 символов
firstName	string	да	Имя участника	1–50 символов
middleName	string	нет	Отчество участника	до 50 символов

Поле	Тип данных	Обязательно	Описание	Ограничения
passportSeries	string	да	Серия паспорта	4 цифры
passportNumber	string	да	Номер паспорта	6 цифр

Таблица 3 – Структура вложенного объекта Participant

Структура ответа ContractResponse (договор и электронные билеты) приведена в таблице 4.

Поле	Тип данных	Обязательно	Описание
contract	object Contract	да	Оформленный договор
clientToken	string	да	Токен клиента для доступа к своим договорам
contract.id	string (uuid)	да	Идентификатор договора
contract.number	string	да	Номер договора
contract.excursionId	string (uuid)	да	Идентификатор экскурсии
contract.totalPrice	number	да	Итоговая стоимость, ₽
contract.status	string (enum)	да	Статус: pending_payment, paid и др.
contract.createdAt	string (date-time)	да	Дата и время создания
contract.paymentDeadline	string (date-time)	да	Срок оплаты договора
contract.participants	array<Participant>	да	Участники договора
contract.tickets	array<Ticket>	да	Электронные билеты участников
ticket.id	string (uuid)	да	Идентификатор билета
ticket.qrCode	string	да	Содержимое QR-кода
ticket.participantName	string	да	ФИО участника
ticket.attended	boolean	да	Отметка явки
ticket.status	string (enum)	да	Статус: active, used, cancelled

Таблица 4 – Структура ответа ContractResponse

Пример JSON-запроса (тело POST /contracts):

```
{
  "excursionId": "e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21",
  "participants": [
    {
      "lastName": "Петров",
      "firstName": "Пётр",
      "middleName": "Петрович",
      "passportSeries": "9701",
      "passportNumber": "123456"
    }
  ],
  "contactEmail": "petrov@example.ru",
  "contactPhone": "+79001234567",
  "consentPersonalData": true
}
```

Листинг 1 – JSON-запрос на оформление договора

Пример успешного JSON-ответа (201 Created):

```
{
  "contract": {
    "id": "8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d",
    "number": "Д-2026-001234",
    "excursionId": "e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21",
    "totalPrice": 1500.00,
    "status": "pending_payment",
    "createdAt": "2026-11-01T12:00:00+03:00",
    "paymentDeadline": "2026-11-01T12:10:00+03:00",
    "participants": [
      { "lastName": "Петров", "firstName": "Пётр", "middleName": "Петрович" }
    ],
    "tickets": [
      {
        "id": "9f8e7d6c-5b4a-3c2d-1e0f-9a8b7c6d5e4f",
        "qrCode": "QR-9F8E7D6C",
        "participantName": "Петров Пётр Петрович",
        "attended": false,
        "status": "active"
      }
    ]
  },
  "clientToken": "ct_7f3a9c2e5b1d"
}
```

Листинг 2 – JSON-ответ при успешном оформлении договора

Пример ошибки валидации (422 Unprocessable Entity):

```
{
  "type": "https://api.excursion-bureau.ru/problems/validation-error",
  "title": "Ошибка валидации",
  "status": 422,
  "detail": "Поле participants должно содержать хотя бы одного участника",
  "instance": "/api/v1/contracts"
}
```

Листинг 3 – JSON-ответ с ошибкой в формате Problem Details

Фрагмент JSON Schema запроса ContractInput (используется в OpenAPI):

```
{
  "type": "object",
  "required": ["excursionId", "participants", "contactEmail", "consentPersonalData"],
  "additionalProperties": false,
  "properties": {
    "excursionId": { "type": "string", "format": "uuid" },
    "participants": {
      "type": "array", "minItems": 1, "maxItems": 20,
      "items": { "$ref": "#/components/schemas/Participant" }
    },
    "contactEmail": { "type": "string", "format": "email" },
    "contactPhone": { "type": "string", "pattern": "^\\+?[0-9]{10,15}$" },
    "consentPersonalData": { "type": "boolean" }
  }
}
```

Листинг 4 – Фрагмент JSON Schema запроса ContractInput

3. Спецификация OpenAPI для одной конечной точки

Спецификация разработана вручную в формате OpenAPI 3.0.3 по восьми шагам учебника OpenAPI и сохранена в файле **8_swagger_ui/openapi.yaml**. Она прошла проверку валидатором openapi-spec-validator (результат: OK) и открывается в Swagger Editor (editor-next.swagger.io).

3.1. Восемь шагов разработки спецификации

Шаг 1. Объект openapi

Указывается версия формата OpenAPI. Для Swagger UI выбрана стабильная версия 3.0.3.

```
openapi: 3.0.3
```

Шаг 2. Объект info

Общая информация об API: название, версия, описание, контакты.

```
info:
  title: API «Экскурсионное бюро» — оформление договора
  version: 1.0.0
  description: |
    Интерактивная документация конечной точки POST /contracts.
    Конечная точка оформляет договор, создаёт электронные билеты
    и возвращает клиентский токен.
```

Шаг 3. Объект servers

Адреса серверов: продуктивный и локальный mock-сервер для тестирования.

```
servers:
  - url: https://api.excursion-bureau.ru/api/v1
    description: Продуктивный сервер
  - url: http://localhost:8000/api/v1
    description: Локальный mock-сервер для тестирования
```

Шаг 4. Объект paths и операция

Описывается ресурс /contracts и операция POST с кратким и подробным описанием.

```
paths:
  /contracts:
    post:
      tags: [Contracts]
      summary: Оформить договор на экскурсию (UC-04)
      description: |
        Создаёт договор, формирует электронные билеты участников
        и возвращает клиентский токен.
      operationId: createContract
```

Шаг 5. Параметры

Для повторных запросов добавлен необязательный заголовок Idempotency-Key.

```
parameters:
  - name: Idempotency-Key
    in: header
    required: false
    description: Ключ идемпотентности повторного запроса (UUID).
    schema:
      type: string
      format: uuid
```

Шаг 6. Тело запроса

Тело запроса ссылается на схему ContractInput и содержит пример.

```
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/ContractInput'
      examples:
        oneParticipant:
          summary: Договор на одного участника
          value:
            excursionId: e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21
            participants:
              - lastName: Петров
                firstName: Пётр
                passportSeries: '9701'
                passportNumber: '123456'
            contactEmail: petrov@example.ru
            consentPersonalData: true
```

Шаг 7. Ответы

Описаны успешный ответ 201, ошибки 400, 409 и 422; для 201 указаны заголовок Location и пример тела.

```
responses:
  '201':
    description: Договор создан, электронные билеты сформированы.
    headers:
      Location:
        schema: { type: string, format: uri }
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ContractResponse'
  '400':
    description: Некорректный запрос
  '409':
    description: Конфликт состояния
  '422':
    description: Ошибка валидации
```

Шаг 8. Компоненты и схемы

Повторно используемые схемы вынесены в components: Participant, ContractInput, Ticket, Contract, ContractResponse, Problem, а также общие ответы об ошибках.

```
components:
  schemas:
    Participant:
      type: object
      required: [lastName, firstName, passportSeries, passportNumber]
      properties:
        lastName: { type: string, maxLength: 50 }
        firstName: { type: string, maxLength: 50 }
        middleName: { type: string, maxLength: 50 }
        passportSeries: { type: string, pattern: '^[0-9]{4}$' }
        passportNumber: { type: string, pattern: '^[0-9]{6}$' }
    ContractInput:
      type: object
      required: [excursionId, participants, contactEmail, consentPersonalData]
      properties:
        excursionId: { type: string, format: uuid }
        participants:
          type: array
          minItems: 1
          maxItems: 20
          items: { $ref: '#/components/schemas/Participant' }
        contactEmail: { type: string, format: email }
        contactPhone: { type: string }
        consentPersonalData: { type: boolean }
    ContractResponse:
      type: object
      required: [contract, clientToken]
      properties:
        contract: { $ref: '#/components/schemas/Contract' }
        clientToken: { type: string }
    Problem:
      type: object
      required: [type, title, status]
      properties:
        type: { type: string, format: uri }
        title: { type: string }
        status: { type: integer }
        detail: { type: string }
        instance: { type: string }
```

3.2. Состав итоговой спецификации

Итоговая спецификация объединяет восемь шагов и содержит следующие разделы.

Раздел OpenAPI	**Содержание**
openapi	Версия формата — 3.0.3
info	Название, версия API 1.0.0, описание, контакты
servers	Продуктивный сервер и локальный mock-сервер
tags	Группа Contracts — оформление договоров
paths	Конечная точка POST /contracts
parameters	Заголовок Idempotency-Key
requestBody	Схема ContractInput и пример запроса
responses	201 ContractResponse; 400, 409, 422 Problem Details
components	Схемы Participant, ContractInput, Ticket, Contract, ContractResponse, Problem

Таблица 5 – Состав спецификации OpenAPI

Полный текст спецификации находится в файле openapi.yaml (объём — около 300 строк). Фрагмент итогового документа с операцией и схемами приведён в листинге 5.

```
paths:
  /contracts:
    post:
      tags: [Contracts]
      summary: Оформить договор на экскурсию (UC-04)
      operationId: createContract
      parameters:
        - name: Idempotency-Key
          in: header
          schema: { type: string, format: uuid }
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ContractInput' }
      responses:
        '201':
          description: Договор создан
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ContractResponse' }
        '422':
          $ref: '#/components/responses/ValidationError'
components:
  schemas:
    ContractInput:
      type: object
      required: [excursionId, participants, contactEmail, consentPersonalData]
      properties:
        excursionId: { type: string, format: uuid }
        participants:
          type: array
          minItems: 1
          maxItems: 20
          items: { $ref: '#/components/schemas/Participant' }
        contactEmail: { type: string, format: email }
        consentPersonalData: { type: boolean }
    ContractResponse:
      type: object
      properties:
        contract: { $ref: '#/components/schemas/Contract' }
        clientToken: { type: string }
```

Листинг 5 – Фрагмент итоговой спецификации OpenAPI

4. Веб-страница с интерактивной документацией Swagger UI

На основе спецификации создана автономная веб-страница Swagger UI. Она находится в папке **8_swagger_ui** и открывается двойным щелчком по файлу index.html: спецификация встроена в страницу, а все файлы Swagger UI сохранены локально, поэтому интернет для просмотра не требуется. Внешний вид страницы приведён на рисунке 1.

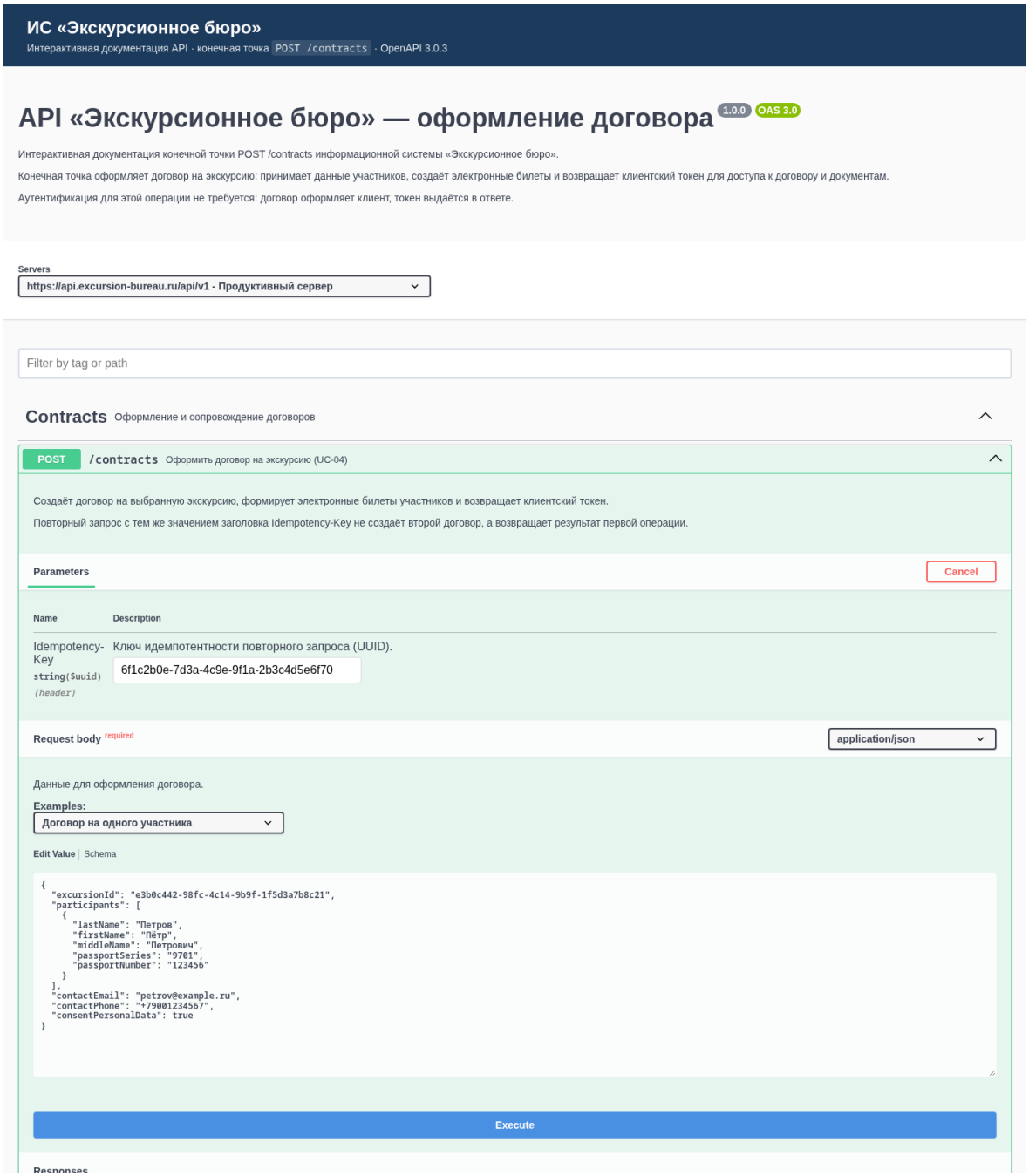


Рисунок 1 – Веб-страница Swagger UI с интерактивной документацией POST /contracts

Файл	Назначение
index.html	Страница Swagger UI со встроенной спецификацией OpenAPI
openapi.yaml	Исходная спецификация OpenAPI 3.0.3 (одна конечная точка)
swagger-ui.css	Стили Swagger UI (дистрибутив swagger-ui-dist 5)
swagger-ui-bundle.js	Основной JavaScript-бандл Swagger UI
swagger-ui-standalone-preset.js	Пресет для StandaloneLayout
mock_server.py	Локальный mock-сервер для проверки Try it out
build_swagger_page.py	Скрипт встраивания спецификации в index.html
README.txt	Инструкция по запуску и обновлению страницы
screenshots/	Скриншоты страницы для отчёта
generated/python-client/	Клиент Python, сгенерированный OpenAPI Generator

Таблица 6 – Состав веб-страницы и вспомогательных файлов

Страница содержит заголовок, выбор сервера, фильтр, группу Contracts и раскрытую операцию POST /contracts. Для операции показаны параметры, тело запроса с примером, кнопка Try it out/Execute, ответы 201, 400, 409, 422 и схемы данных.

4.1. Запуск и настройка

Для просмотра достаточно открыть index.html в браузере. Для выполнения запросов через Try it out используется локальный mock-сервер:

```
cd 8_swagger_ui
python3 mock_server.py
```

После запуска mock-сервера в Swagger UI выбирается сервер «http://localhost:8000/api/v1 — Локальный mock-сервер для тестирования», редактируется тело запроса и нажимается Execute. Mock-сервер реализует POST /api/v1/contracts, проверяет обязательные поля и возвращает договор с электронными билетами.

5. Тестирование и отладка API через Swagger UI

Интерактивная консоль Swagger UI позволяет отправлять запросы, изменять тело и заголовки, просматривать ответ, заголовки и длительность выполнения. В таблице 7 приведены проверенные сценарии. На рисунке 2

показан результат выполнения запроса через Try it out: Swagger UI отобразил сгенерированную команду curl, URL запроса, ответ 201 с договором и билетами, заголовки ответа и длительность выполнения (21 мс).

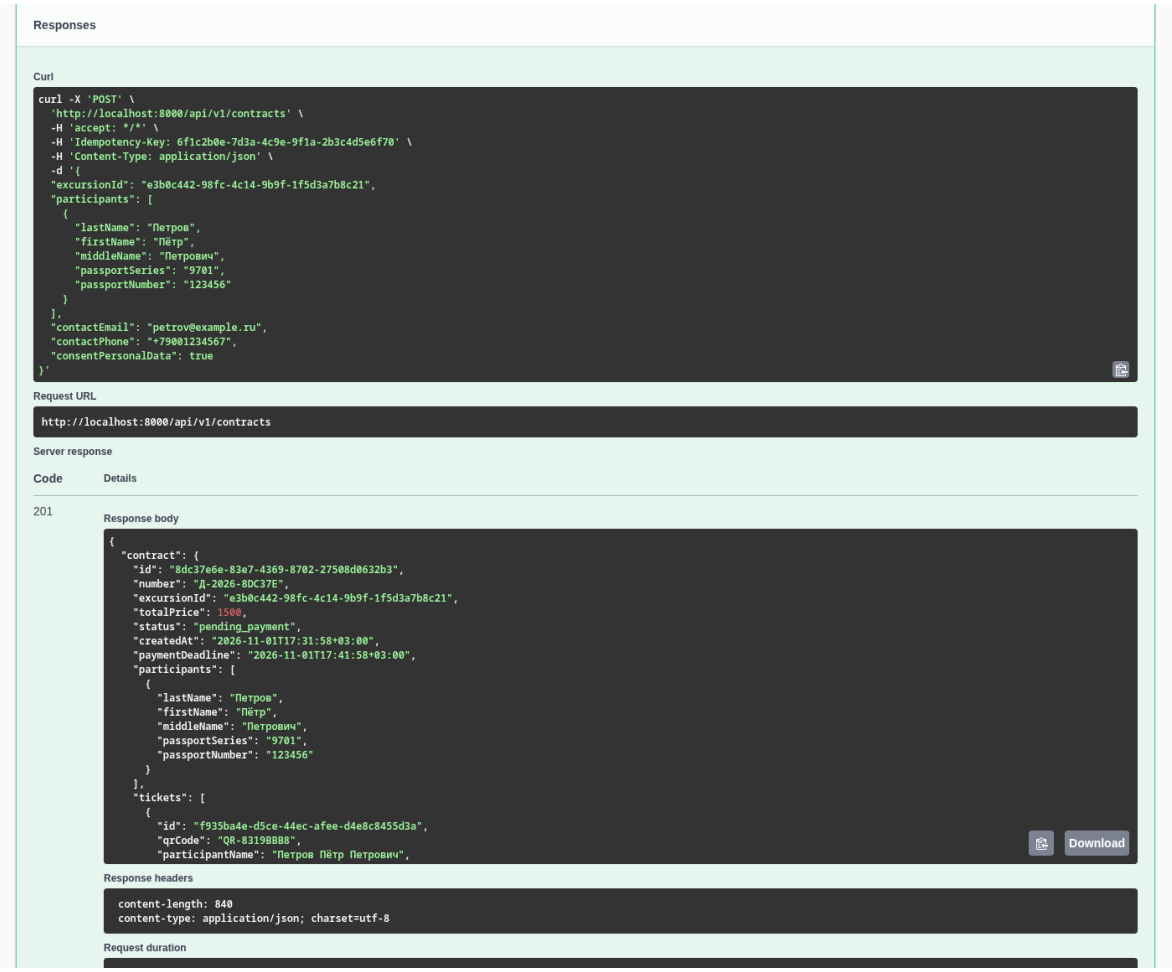


Рисунок 2 – Ответ 201 в интерактивной консоли Swagger UI после выполнения запроса

№	Сценарий	Запрос	Ожидаемый код	Фактический результат
1	Успешное оформление договора	POST /contracts с валидным ContractInput	201	201; получены договор, билеты и clientToken
2	Пустой список участников	POST /contracts, participants: []	422	422; сообщение об ошибке валидации
3	Некорректный JSON	POST /contracts, тело {bad	400	400; ошибка формата JSON
4	Неизвестный путь	POST /api/v1/unknown	404	404; ресурс не найден
5	Нет свободных мест (конфликт)	POST /contracts при отсутствии мест	409	Описано в спецификации; mock-сервер не моделирует состояние

Таблица 7 – Сценарии тестирования конечной точки

Эквивалентный запрос можно выполнить командой curl:

```
curl -X POST http://localhost:8000/api/v1/contracts \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 6f1c2b0e-7d3a-4c9e-9f1a-2b3c4d5e6f70" \
  -d @contract-request.json
```

Для проверки ошибок достаточно изменить тело запроса: убрать участников (422), передать некорректный JSON (400) или указать несуществующий путь (404).

6. Генерация кода по спецификации

По спецификации OpenAPI сгенерирован клиент на Python с помощью OpenAPI Generator (поддерживаемый форк Swagger Codegen), версия 7.26.0:

```
npx @openapitools/openapi-generator-cli generate \
  -i openapi.yaml -g python -o generated/python-client \
  --additional-properties=packageName=excursion_api
```

Генератор создал пакет excursion_api с моделями данных, API-классом ContractsApi, HTTP-клиентом, конфигурацией, тестами и документацией (таблица 8).

Сгенерированный файл	**Назначение**
excursion_api/api/contracts_api.py	Метод create_contract для POST /contracts
excursion_api/models/contract_input.py	Модель запроса ContractInput
excursion_api/models/participant.py	Модель участника Participant
excursion_api/models/contract.py	Модель договора Contract
excursion_api/models/ticket.py	Модель электронного билета Ticket
excursion_api/models/problem.py	Модель ошибки Problem Details
docs/ContractsApi.md	Документация по сгенерированному API
test/test_contract_input.py	Заготовки тестов модели

Таблица 8 – Состав сгенерированного клиента

Фрагмент сгенерированной модели ContractInput:

```
class ContractInput(BaseModel):
    excursion_id: UUID = Field(..., alias="excursionId")
    participants: Annotated[List[Participant], Field(min_length=1, max_length=20)]
    contact_email: StrictStr = Field(..., alias="contactEmail")
    contact_phone: Optional[str] = Field(default=None, alias="contactPhone")
    consent_personal_data: StrictBool = Field(..., alias="consentPersonalData")

    @field_validator('contact_phone', mode="before")
    def contact_phone_validate_regular_expression(cls, value):
        if value is None:
            return value
        if not re.match(r"^\+?[0-9]{10,15}$", value):
```

```
            raise ValueError("must validate the regular expression")
        return value
```

Листинг 6 – Фрагмент сгенерированной модели ContractInput

Сгенерированный код является заготовкой: он корректно отражает схемы и методы, но требует настройки сериализации, аутентификации, повторных запросов и обработки ошибок под конкретное приложение. Для учебной демонстрации возможностей OpenAPI этого достаточно.

7. Ответы на вопросы

1. Что означает «API-first подход»?

Это подход, при котором сначала разрабатывается и фиксируется контракт API (спецификация), а уже затем реализуется сервер и клиенты. При расхождении контракта и реализации изменяют реализацию, а не контракт.

2. Назовите типы веб-API.

REST, SOAP, GraphQL, gRPC, WebSocket, JSON-RPC/XML-RPC; по аудитории — публичные, партнёрские и внутренние (private); по способу применения — клиентские и серверные.

3. Что означает слово REST?

Representational State Transfer — «передача состояния представления»; архитектурный стиль взаимодействия компонентов распределённой системы.

4. Что такое веб-сервис? Какой тип API использует веб-сервис?

Веб-сервис — программная служба, доступная по сети через стандартизированный интерфейс. Чаще всего используется REST (RESTful веб-сервис) или SOAP.

5. Какие функции выполняет API-шлюз?

Маршрутизацию запросов, аутентификацию и авторизацию, ограничение скорости, кэширование, трансляцию протоколов, агрегацию

ответов, логирование и мониторинг, управление версиями и балансировку нагрузки.

6. Почему REST не является протоколом?

REST — это набор архитектурных ограничений (стиль), а не протокол передачи данных. Он не определяет формат сообщений и транспорт; обычно используется HTTP, но сам стиль может применяться и с другими протоколами.

7. Почему архитектура, основанная на событиях, не будет удовлетворять требованиям RESTful?

Событийная архитектура асинхронна и основана на публикации/подписке, а REST предполагает клиент-инициированные запросы-ответы, отсутствие состояния на сервере и единообразный интерфейс. Постоянные соединения и push-уведомления нарушают stateless-модель и единообразие интерфейса.

8. Назовите основные принципы REST.

Клиент-сервер, отсутствие состояния (stateless), кэшируемость, единообразный интерфейс, многоуровневая система, код по требованию (необязательно).

9. Какую цель преследует требование «RESTful-система должна производить операции в клиент-серверной модели»?

Разделение ответственности: клиент отвечает за пользовательский интерфейс, сервер — за данные и бизнес-логику. Это позволяет развивать части независимо и повышает масштабируемость.

10. В чем смысл требования «Отсутствие состояния» в RESTful-системе?

Сервер не хранит состояние сессии клиента между запросами: каждый запрос содержит всю необходимую информацию (токен, параметры). Это упрощает масштабирование, восстановление после сбоев и кэширование.

11. Принцип унификации или единообразия интерфейса. Как достигается унификация?

За счёт единых правил идентификации ресурсов (URI), использования стандартных HTTP-методов и кодов состояния, единых форматов представления (JSON/XML) и гипермедиа-ссылок.

12. Назовите четыре составляющих унифицированного интерфейса.

Идентификация ресурсов; манипулирование ресурсами через представления; самоописываемые сообщения; гипермедиа как движок состояния приложения (HATEOAS).

13. Как реализуется унификация интерфейса в REST?

Ресурсы адресуются URI, действия выполняются стандартными HTTP-методами (GET, POST, PUT, PATCH, DELETE), ответы используют единые коды состояния и медиа-типы, а связи между ресурсами выражаются гиперссылками.

14. Почему возникла необходимость разработки ключа API? В чем недостатки базовой аутентификации?

Ключ API позволяет идентифицировать клиентское приложение, ограничивать доступ и квоты, не передавая логин и пароль. Базовая аутентификация передаёт учётные данные в каждом запросе (base64), не даёт гранулярных прав, сложнее в ротации и при компрометации пароля требует его смены.

15. Есть ли альтернативы REST?

Да: GraphQL, gRPC, SOAP, WebSocket, JSON-RPC/XML-RPC, событийные архитектуры и брокеры сообщений (Kafka, RabbitMQ), а также файловый обмен и очереди задач.

8. Заключение

В ходе лабораторной работы выполнено документирование веб-API информационной системы «Экскурсионное бюро»:

– сформированы JSON-структуры данных для конечной точки оформления договора: таблицы полей запроса и ответа, примеры успешного ответа и ошибки, фрагмент JSON Schema;

– вручную разработана спецификация OpenAPI 3.0.3 для одной конечной точки POST /contracts по восьми шагам; спецификация прошла валидацию openapi-spec-validator;

– создана автономная веб-страница с интерактивной документацией Swagger UI; спецификация встроена в страницу, файлы Swagger UI сохранены локально;

– проведено тестирование через Try it out с локальным mock-сервером: успешный запрос возвращает 201, ошибки валидации — 422, некорректный JSON — 400, неизвестный путь — 404;

– по спецификации сгенерирован клиент Python с помощью OpenAPI Generator 7.26.0.

Полученные артефакты — отчёт, спецификация openapi.yaml, страница Swagger UI, mock-сервер и сгенерированный клиент — находятся в папке «8_swagger_ui».